

Hybrid GCP Governance Engine Implementation

Prepared for: Cloud Engineering | July 2026

Executive Summary: The current environment relies on a mixed provisioning model (Terraform, GCP Console, gcloud CLI). Deploying the custom Cloud Run compliance engine natively across this environment introduces the risk of **Infrastructure as Code (IaC) state conflicts**. To mitigate this, we are transitioning to a *Label-Driven State Management* approach combined with a centralized *BigQuery-backed Dashboard*.

1. Challenge: IaC State Conflicts in a Mixed Environment

The core challenge with the existing custom Python engine is maintaining state synchronization. If the engine actively remediates a Terraform-managed resource (e.g., disabling a public IP via `cloudrun/clients/sql.py`), the next `terraform apply` run will detect this change as drift and overwrite it. We require a mechanism that allows the custom engine to track and enforce compliance without fighting the IaC state files.

2. Recommended Implementation: Label-Driven Hybrid Engine

We will orchestrate native GCP services alongside the custom Python engine to achieve comprehensive coverage without state conflicts.

Layer 1: Universal Discovery (Cloud Asset Inventory)

Mechanism	What It Does
CAI Real-Time Feeds	Cloud Asset Inventory (CAI) replaces custom API polling. It pushes real-time configuration changes to Pub/Sub.
Pub/Sub Ingress	The engine listens to these feeds via <code>cloudrun/ingress/pubsub.py</code> , ensuring immediate visibility into Greenfield resource creation regardless of the provisioning method.

Layer 2: Label-Driven Compliance Tracking

To avoid Terraform drift, all Terraform modules will be updated to include the `lifecycle { ignore_changes = [labels] }` block. This carves out a safe operational space for the engine to write state directly to the resource metadata.

- **Violation Tagging:** The engine appends `compliance-status: violation` rather than altering core resource configurations.

- **Escalation Tracking:** The engine automatically cycles labels (e.g., `action-required: 7-days` to `action-required: overdue`) to manage grace periods.

3. Remediation Matrix (Bifurcated Enforcement)

The engine will execute distinct remediation workflows based on resource provenance tags.

Resource Origin	Engine Action	Resolution Path
Terraform-Managed <i>(Label: <code>provisioned-by: terraform</code>)</i>	Tag & Alert Only	Engine applies violation labels and alerts the owner. Developer must submit a Pull Request to fix the Terraform configuration.
Console / CLI <i>(Missing provenance tag)</i>	Auto-Remediate	If <code>auto-remediate: true</code> is defined in the registry, the engine actively reverts the configuration via native clients.

4. Visibility: BigQuery & Dashboard Reporting

To combat alert fatigue, compliance visibility will be centralized into a frontend dashboard.

- **Data Aggregation:** The `reporting.py` service logs all CAI evaluations, label updates, and violations into BigQuery.
- **Dashboard Frontend:** Using the existing `cloudrun/templates/dashboard.html` and `dashboard.js`, engineering teams can filter violations by team, resource type, and severity.
- **Gamification & Accountability:** The dashboard will highlight compliance scores per team, driving decentralized remediation.